U·tad

Unidad 2 — JavaScript avanzado y herramientas del frontend

Web Development I, the Client · 3.º de Ingeniería del Software · 2026–2027

Arrays y objetos · Inmutabilidad · `this`, closures y clases · Event loop, promesas y `async/await` · `fetch` · Módulos, npm, Vite y TypeScript · Cuaderno de ejercicios 2 · Misión M2 «Async Odyssey»

Al terminar esta unidad sabrás…

- Transformar datos con `map`, `filter`, `reduce`, `groupBy` y los métodos **inmutables** (`toSorted`, `with`), sin bucles innecesarios.
- Explicar la diferencia entre **referencia y valor** y hacer copias superficiales y profundas (`...`, `structuredClone`).
- Predecir qué vale `this`, usar closures para **encapsular** estado y escribir clases con campos privados.
- Dibujar el **event loop** y justificar el orden de salida de un código con `setTimeout` y promesas.
- Escribir código asíncrono con `async/await` y `try/catch`, en paralelo cuando toca (`Promise.all` / `allSettled`).
- Consumir una API con `fetch`, comprobar `respuesta.ok` y mostrar los **tres estados**: cargando, error y datos.
- Organizar un proyecto en **módulos ES** con npm y **Vite**, y tipar sus fronteras con **TypeScript**.

Mapa de la unidad

1. Arrays en profundidad
2. Objetos en profundidad
3. Referencia vs valor; copias
4. `this`, `call`, `apply`, `bind`
5. Closures y patrones
6. Prototipos y clases ES
7. `Map` y `Set`
8. Módulos ES y manejo de errores
9. JSON y Web APIs del navegador
10. **Asincronía:** *event loop*, promesas, `async/await`
11. La API `fetch`
12. Herramientas: npm, Vite y TypeScript
13. Buenas prácticas
14. **Reto jefe:** Atlas multiverso · **Arena:** misión M2

> Ejercicios del cuaderno 2 intercalados: 4 calentamientos 🔴, 3 misiones 🟡, 1 reto jefe 🟢.

1 · Arrays en profundidad

`at()`, mutación frente a inmutabilidad, y los métodos que sustituyen a los bucles.

Mutar o no mutar: la distinción de la unidad

```
const letras = ["a", "b", "c", "d"];
letras.at(-1);  // "d": el último, sin arr[arr.

// Métodos de MUTACIÓN: alteran el array origin
const pila = [1, 2, 3];
pila.push(4);          // [1, 2, 3, 4]
pila.pop();            // [1, 2, 3]
pila.unshift(0);       // [0, 1, 2, 3]
pila.shift();          // [1, 2, 3]
pila.splice(1, 1, 9); // [1, 9, 3]
```

Mutación	Equivalente inmutable (ES2023+)
`sort()`	`toSorted()`
`reverse()`	`toReversed()`
`splice()`	`toSpliced()`
`arr[i] = x`	`with(i, x)`

⚠️ `sort()` sin comparador ordena como **texto**: `[10, 2, 1].sort()` → `[1, 10, 2]`. Para números, `sort((a, b) => a - b)`.

💡 Prefiere los inmutables salvo razón concreta: código predecible y encaja con React (U3).

DEMO EN VIVO

Métodos inmutables: el original no cambia

```
// Métodos inmutables: el original NO cambia
const original = [3, 1, 2];

const ordenado = original.toSorted((a, b) => a - b); // [1, 2, 3]
const invertido = original.toReversed();             // [2, 1, 3]
const cambiado = original.with(0, 99);               // [99, 1, 2]

console.log("ordenado:", ordenado);
console.log("original:", original); // [3, 1, 2]  ← intacto

// Ahora compáralo con la versión que muta:
const otro = [3, 1, 2];
otro.sort((a, b) => a - b);
console.log("tras sort(), el original:", otro); // ← ya no es [3, 1, 2]
```

DEMO EN VIVO

`map`, `filter`, `reduce` y compañía

```
const productos = [
  { nombre: "Teclado", precio: 25, stock: 0 },
  { nombre: "Ratón",   precio: 15, stock: 12 },
  { nombre: "Monitor", precio: 180, stock: 4 },
];
const nombres = productos.map(p => p.nombre);            // transforma → mismo tamaño
const disponibles = productos.filter(p => p.stock > 0);  // selecciona
const valorTotal = productos.reduce((acc, p) => acc + p.precio * p.stock, 0); // acumula
const monitor = productos.find(p => p.precio > 100);     // el primero que cumple
const hayCaros = productos.some(p => p.precio > 100);    // boolean (every: todos)

console.log("nombres:", nombres, "· disponibles:", disponibles.map(p => p.nombre));
console.log("valor total del stock:", valorTotal, "· ¿hay caros?:", hayCaros);
console.log("pipeline:", productos.filter(p => p.stock > 0).map(p => p.nombre).toSorted());
```

Ahora tú · 1.1 El podio inmutable

⏱ **10 min** · 🔴 **sin IA** · cuaderno 2, nivel 1

```
const puntuaciones = [880, 1250, 445, 990, 1250, 720];
```

Sin modificar `puntuaciones` en ningún momento:

- (a) `podio`: las **tres mejores puntuaciones únicas**, de mayor a menor.
- (b) `corregidas`: igual que el original pero con la más baja sustituida por `500`.
- Verifica al final que `puntuaciones` sigue intacto.

```
podio      → [1250, 990, 880]
corregidas → [880, 1250, 500, 990, 1250, 720]
original   → [880, 1250, 445, 990, 1250, 720]
```

> 💡 Duplicados fuera: `[...new Set(puntuaciones)]` (lo veremos en el bloque 7). `toSorted` para ordenar sin mutar, `indexOf` del mínimo y `with(indice, valor)`.

2 · Objetos en profundidad

Shorthand, propiedades computadas, desestructuración, *spread/rest*, `Object.*` y `groupBy`.

Shorthand, propiedades computadas y desestructuración

```
const nombre = "Ana";
const edad = 30;

// Shorthand: la variable se llama igual que la c
const persona = { nombre, edad };

// Computed properties: clave dinámica
const clave = "rol";
const config = { [clave]: "admin", [`nivel_${edad
// { rol: "admin", nivel_30: true }

const usuario = { direccion: null };
usuario.direccion?.ciudad ?? "desconocida"; // U1
```

```
// De objetos: renombrar y valores por defecto
const { nombre: n, edad: e = 0, ciudad = "Madrid"
// n = "Ana", e = 30, ciudad = "Madrid"

// De arrays (por posición)
const [primero, , tercero] = ["a", "b", "c"];

// En parámetros de función (muy común)
function crearTarjeta({ titulo, color = "azul" })
  return `${titulo} (${color})`;
}
crearTarjeta({ titulo: "Inicio" }); // "Inicio (a
```

Spread y rest: expandir o agrupar

```
// Spread: copiar y combinar
const base = { tema: "oscuro", idioma: "es" };
const ampliado = { ...base, idioma: "en", fuente:
// { tema: "oscuro", idioma: "en", fuente: 14 }
// ← lo de la derecha sobreescribe

const a = [1, 2], b = [3, 4];
const unidos = [...a, ...b]; // [1, 2, 3, 4]
```

```
// Rest: recoger el resto
const { tema, ...otras } = ampliado;
// tema = "oscuro", otras = { idioma, fuente }


function suma(...numeros) {   // rest en parámetr
  return numeros.reduce((t, n) => t + n, 0);
}
suma(1, 2, 3, 4); // 10
```

> El mismo `...` **expande** (spread) o **agrupa** (rest) según el contexto. `{ ...obj, campo: nuevo }` es la forma idiomática de «actualizar sin mutar».

Métodos estáticos de `Object`

```
const punto = { x: 1, y: 2 };

Object.keys(punto);    // ["x", "y"]
Object.values(punto);  // [1, 2]
Object.entries(punto); // [["x", 1], ["y", 2]]  → se puede map/filter y…

// …reconstruir desde entries (útil tras filtrar/transformar)
Object.fromEntries([["a", 1], ["b", 2]]); // { a: 1, b: 2 }

// Combinar (muta el primer argumento) — preferible el spread
Object.assign({}, base, { idioma: "fr" });

// Congelar: impide añadir, borrar o modificar propiedades
const constante = Object.freeze({ PI: 3.1416 });
constante.PI = 3; // ❌ TypeError en strict mode
```

DEMO EN VIVO

`Object.groupBy` (ES2024): agrupar sin `reduce`

```
const inventario = [
  { nombre: "manzana", tipo: "fruta" },
  { nombre: "zanahoria", tipo: "verdura" },
  { nombre: "plátano", tipo: "fruta" },
];

const porTipo = Object.groupBy(inventario, item => item.tipo);
// {
//   fruta:   [{manzana}, {plátano}],
//   verdura: [{zanahoria}]
// }

console.log(porTipo);
console.log("frutas:", porTipo.fruta.map(f => f.nombre));
```

> También `Map.groupBy`: igual, pero devuelve un `Map` (útil si la clave de agrupación no es un string).

Ahora tú · 1.2 Desempaquetando la API

⏱ **10 min** · 🔴 **sin IA** · cuaderno 2, nivel 1

```
const respuesta = {
  data: {
    usuario: {
      nombre: "Río", nivel: 12,
      stats: { victorias: 40, derrotas: 5 },
    },
    amigos: ["Vega", "Kai", "Nova"],
  },
};
```

Con **una sola sentencia** de desestructuración extrae:

- `nombre`
- `victorias` renombrada a `wins`
- `rango` con valor por defecto `"novato"` (no existe)
- el primer amigo como `mejorAmigo` y el resto en `resto`

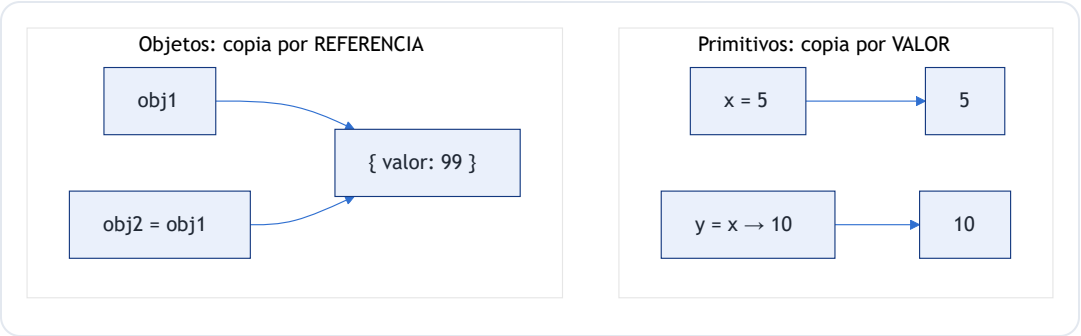
```
Río · wins: 40 · rango: novato · mejor amigo: Vega · r
```

> 💡 Se anidan patrones: `const { data: { usuario: { nombre, stats: { victorias: wins } } } } = respuesta;` y `amigos: [mejorAmigo, ...resto]`.

3 · Referencia vs valor; copia superficial vs profunda

Por qué cambiar una copia cambia el original, y cómo evitarlo.

Primitivos por valor, objetos por referencia




```
// Por valor: copia independiente
let x = 5;
let y = x;
y = 10;
console.log(x); // 5  ← no afectado

// Por referencia: dos nombres, UN objeto
const obj1 = { valor: 1 };
const obj2 = obj1;
obj2.valor = 99;
console.log(obj1.valor); // 99  ← ¡afectado!
```

- `y = x` copia el número: dos cajas. `obj2 = obj1` copia **la flecha**, no la caja.
- Lo mismo al pasar un objeto a una función: dentro se toca **el mismo** objeto.

DEMO EN VIVO

Copia superficial (`...`) y copia profunda (`structuredClone`)

```
const original = { nombre: "Eva", config: { tema: "claro" }, fecha: new Date() };

const copia = { ...original };     // copia SUPERFICIAL: solo el primer nivel
copia.nombre = "Mar";              // afecta solo a copia
copia.config.tema = "oscuro";      // ¡afecta también a original! (anidado, misma referencia)
console.log("tras el spread, original.config.tema:", original.config.tema); // "oscuro"

const clon = structuredClone(original); // copia PROFUNDA: anidados, Date, Map, Set…
clon.config.tema = "claro";
console.log("tras structuredClone, original.config.tema:", original.config.tema); // sigue "oscuro"
console.log("la fecha del clon sigue siendo Date:", clon.fecha instanceof Date);
```

> ⚠️ El viejo truco `JSON.parse(JSON.stringify(obj))` pierde funciones y `undefined`, convierte `Date` en string y falla con referencias circulares. Usa `structuredClone`.

4 · `this`, `call`, `apply`, `bind`

`this` se decide **al llamar**, no al definir.

`this`: quién está a la izquierda del punto

```
const coche = {
  marca: "Tesla",
  describir() {
    return `Coche: ${this.marca}`; // this = coche
  },
};
coche.describir(); // "Coche: Tesla"

// Pérdida de contexto: al extraer el método…
const f = coche.describir;
f(); // ❌ TypeError: this es undefined (strict mode)

// Arrow: NO tiene this propio, hereda el de fuera
const temporizador = {
  segundos: 0,
  iniciar() {
    setInterval(() => { this.segundos++; }, 1000); // this = temporiz
  },
};
```

```
function presentar(saludo, signo) {
  return `${saludo}, soy ${this.nombre}${signo}`;
}
const ana = { nombre: "Ana" };

presentar.call(ana, "Hola", "!");    // argumentos sueltos
presentar.apply(ana, ["Hola", "!"]); // argumentos en array
const saludarAna = presentar.bind(ana); // nueva función con this fij
saludarAna("Buenas", ".");           // "Buenas, soy Ana."

// La cura de la pérdida de contexto:
const g = coche.describir.bind(coche);
g(); // "Coche: Tesla"
```

> `call` / `apply` **ejecutan ya**; `bind` **devuelve** una función que recordará su `this`.

5 · Closures y patrones

Una función que **recuerda** el ámbito donde nació: la base de la encapsulación en JavaScript.

DEMO EN VIVO

Closure: estado privado de verdad

```
function crearContador() {
  let cuenta = 0;            // variable "privada" capturada por el closure
  return {
    incrementar() { return ++cuenta; },
    valor() { return cuenta; },
  };
}

const contador = crearContador();
contador.incrementar(); // 1
contador.incrementar(); // 2

console.log("valor:", contador.valor());              // 2
console.log("¿se ve la variable?:", contador.cuenta); // undefined ← encapsulación real

// Cada llamada a la factory crea su propio ámbito independiente
```

Patrones con closures: *factory* y módulo

```
// Factory: crea objetos sin new ni clases
function crearUsuario(nombre, rol) {
  return {
    nombre,
    rol,
    puedeEditar: () => rol === "admin",
  };
}
const editor = crearUsuario("Leo", "admin");
editor.puedeEditar(); // true
```

```
// Módulo (IIFE): estado privado + API pública
const carrito = (function () {
  const items = [];                          // estado p
  return {
    add(item) { items.push(item); },
    total() { return items.length; },
  };
})();
carrito.add("libro");
carrito.total(); // 1
```

> El patrón módulo es lo que los **módulos ES** (bloque 8) hacen por ti: ámbito propio por fichero y solo se exporta lo que decides.

6 · Prototipos y clases ES

`class` es azúcar sintáctico sobre la herencia por prototipos: más legible, mismo motor.

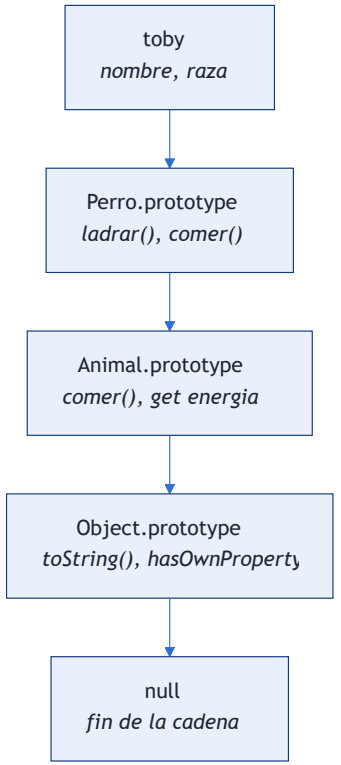
Una clase moderna: campos privados, getters, estáticos

```
class Animal {
  #energia = 100;          // campo privado (solo accesible dentro de la clase)
  static especies = 0;     // propiedad estática (de la clase, no de instancias)

  constructor(nombre) {
    this.nombre = nombre;
    Animal.especies++;
  }
  comer() {                // método de instancia (vive en el prototipo)
    this.#energia += 10;
    return `${this.nombre} come`;
  }
  get energia() { return this.#energia; }              // getter
  set energia(v) { this.#energia = Math.max(0, v); }   // setter
  static crearAnonimo() { return new Animal("???"); }  // método estático
}
const gato = new Animal("Misi");
gato.comer();     // "Misi come"
gato.energia;     // 110 (getter) · Animal.especies → 1
```

Herencia con `extends` y `super`; la cadena de prototipos

```
class Perro extends Animal {
  constructor(nombre, raza) {
    super(nombre);   // obligatorio antes de this
    this.raza = raza;
  }
  ladrar() { return `${this.nombre}: ¡Guau!`; }
  comer() {
    return super.comer() + " con ganas";
  }
}
const toby = new Perro("Toby", "Labrador");
toby.comer();           // "Toby come con ganas"
toby instanceof Animal; // true
```



Al pedir `toby.comer()` se sube por la cadena hasta encontrarlo; `super.comer()` salta un eslabón. `__proto__` es legado: usa `Object.getPrototypeOf()`.

7 · `Map` y `Set`

Diccionarios con claves de cualquier tipo y colecciones sin duplicados.

`Map`: pares clave-valor sin las limitaciones del objeto

```
const cache = new Map();
cache.set("usuario", { id: 1 });
cache.set(42, "respuesta");   // clave numérica
cache.get("usuario");         // { id: 1 }
cache.has(42);                // true
cache.size;                   // 2
for (const [clave, valor] of cache) {
  /* itera en orden de inserción */
}

// Acumular: contar apariciones
recuento.set(nombre, (recuento.get(nombre) ?? 0
```

Usa…	cuando…
Objeto	claves string fijas y conocidas, datos estructurados
`Map`	claves dinámicas o de cualquier tipo, muchas altas/bajas, necesitas `size` u orden
Array	colección ordenada con posibles duplicados
`Set`	valores únicos, comprobar pertenencia rápido

> La **caché** de la Misión 2 y del reto jefe es un `Map` con la URL como clave.

`Set`: valores únicos (y operaciones de conjuntos)

```
const etiquetas = new Set(["js", "css", "js", "html"]);
etiquetas.size;        // 3  (el "js" duplicado se descarta)
etiquetas.add("react");
etiquetas.has("css");  // true

// Truco habitual: eliminar duplicados de un array
const sinDuplicados = [...new Set([1, 1, 2, 3, 3])]; // [1, 2, 3]

console.log("tamaño tras añadir 'react':", etiquetas.size); // 4
console.log("sin duplicados:", sinDuplicados);

// Baseline 2024–2025: union, intersection, difference
const frontend = new Set(["js", "css", "html"]);
const backend = new Set(["js", "sql"]);
console.log("intersección:", [...frontend.intersection(backend)]); // ["js"]
console.log("diferencia:", [...frontend.difference(backend)]);     // ["css", "html"]
```

Ahora tú · 1.3 Recuento electoral con `Map` y `Set`

⏱ **15 min** · 🔴 **sin IA** · cuaderno 2, nivel 1

```
const votos = ["Ana", "Luis", "Ana", "Sara", "Luis", "Ana", "Sara", "Ana"];
```

`escrutinio(votos)` devuelve `{ candidatos, recuento, ganadora }`:

- `candidatos`: nombres únicos (con `Set`).
- `recuento`: `Map` nombre → número de votos.
- `ganadora`: quien tiene más votos.

```
candidatos → ["Ana", "Luis", "Sara"]
recuento   → Map(3) { "Ana" => 4, "Luis" => 2,
ganadora   → "Ana"
```

> 💡 Acumula con `recuento.set(nombre, (recuento.get(nombre) ?? 0) + 1)`. Para la ganadora, itera las *entries* guardando el máximo, o `[...recuento].toSorted(...)`.

8 · Módulos ES y manejo de errores

Ficheros con ámbito propio, `import` / `export`, y errores que se lanzan como objetos.

Módulos ES: `export`, `import` e `import()` dinámico

```
// matematicas.js
export const PI = 3.1416;                    // con nombre
export function area(r) { return PI * r * r; }
export default function saludar() { /* … */ } // por d
```

```
// app.js
import saludar, { PI, area } from "./matematicas.js";
import { area as calcularArea } from "./matematicas.js
import * as Mate from "./matematicas.js";
```

```
// Import dinámico: carga bajo demanda (devuelve una p
async function cargarGraficos() {
  const modulo = await import("./graficos.js");
  modulo.dibujar();
}
```

```
<script type="module" src="app.js"></script>
```

- Cada módulo tiene **su propio ámbito**: nada se filtra al global.
- *Strict mode* y `defer` automáticos.
- El navegador solo carga módulos por **HTTP**, no desde `file://`: usa el servidor de VS Code o Vite.

`throw`, `try/catch/finally` y errores personalizados

```
function dividir(a, b) {
  if (b === 0) throw new Error("División por cero");
  return a / b;
}

try {
  dividir(10, 0);
} catch (error) {
  console.error(error.message); // "División por cero"
} finally {
  console.log("Operación intentada"); // siempre
}
```

> Lanza siempre **instancias de** `Error` (o subclases), nunca strings: tienen `message`, `name` y *stack trace*.

```
class ErrorValidacion extends Error {
  constructor(campo) {
    super(`El campo "${campo}" no es válido`);
    this.name = "ErrorValidacion";
    this.campo = campo;
  }
}

try {
  throw new ErrorValidacion("email");
} catch (e) {
  if (e instanceof ErrorValidacion) {
    console.error(`Validación: ${e.campo}`);
  }
}
```

9 · JSON y Web APIs del navegador

Lo que el navegador añade al lenguaje: `localStorage`, temporizadores, `location` e `history`.

JSON y `localStorage`: el navegador solo guarda texto

```
const objeto = { nombre: "Sara", activo: true, roles: ["

// Objeto → texto JSON
const texto = JSON.stringify(objeto);
// '{"nombre":"Sara","activo":true,"roles":["editor"]}'
const bonito = JSON.stringify(objeto, null, 2); // inden

// Texto JSON → objeto
const recuperado = JSON.parse(texto);
recuperado.nombre; // "Sara"
```

> ⚠️ `JSON.parse` **lanza** si el texto no es válido: `try/catch` cuando venga de fuera (red, `localStorage`, usuario).

```
// Guardar (solo strings: serializa con JSON)
localStorage.setItem("tema", "oscuro");
localStorage.setItem("usuario", JSON.stringify({ id: 1, r

// Leer
const tema = localStorage.getItem("tema");
const usuario = JSON.parse(localStorage.getItem("usuario

// Borrar
localStorage.removeItem("tema");
localStorage.clear();
```

- `localStorage` persiste al cerrar; `sessionStorage` muere con la pestaña.
- **Nunca** datos sensibles: lo lee cualquier JS de la página. Se inspecciona en *Application*.

Temporizadores, `location` e `history`

```
// setTimeout: una vez tras N ms (devuelve un id)
const id = setTimeout(() => console.log("3 s después
clearTimeout(id); // cancela si aún no se ha ejecut

// setInterval: repite cada N ms
const tic = setInterval(() => console.log("tic"), 1
clearInterval(tic); // detiene la repetición
```

> Todo `setInterval` necesita su `clearInterval`: en React (U3) será la «limpieza» de un efecto.

```
location.href;              // URL completa actual
location.pathname;          // "/productos/42"
location.href = "/inicio";  // navegar (recarga)
location.reload();

history.back();             // como el botón «atrás»
history.pushState({}, "", "/nueva-ruta");
// cambia la URL SIN recargar: base del enrutado SPA
```

- `history.pushState` es la pieza sobre la que se apoyan los enrutadores de React y Next.js.
- Otras Web APIs: `navigator`, `URL` / `URLSearchParams`, `FormData` (U4), `fetch` (bloque 11).

Misión 1 · «Aurora FM»: el gestor de playlists inmutable 📻

🟡 **IA como pair** · ⏱ **90 min** · cuaderno 2, nivel 2

- 🎯 **Objetivo:** un módulo de playlist donde **ninguna función modifique los datos originales**, y que sobreviva a un cierre del navegador.
- 🧰 **Practicas:** métodos inmutables, `map` / `filter` / `reduce`, `Object.groupBy`, `structuredClone`, JSON y `localStorage`.

```
// playlist.js — datos de partida (5 canciones)
export const cancionesIniciales = [
  { id: 1, titulo: "Midnight City", artista: "M83",
    genero: "synthpop", duracion: 243, favorita: false },
  // …
];
```

Fases

1. **Operaciones básicas:** `agregar`, `eliminar`, `alternarFavorita` → devuelven un **array nuevo**.
2. **Consultas:** `duracionTotal` (`"mm:ss"`, con `reduce`), `ordenarPor` (`toSorted`), `porGenero` (`Object.groupBy`).
3. **Instantáneas:** `crearInstantanea` con `structuredClone`; demuestra en consola que la copia es independiente.
4. **Persistencia:** `guardar` / `cargar` en `localStorage` (clave `"aurora-fm"`); JSON corrupto → `cancionesIniciales`, con `try/catch`.
5. **Demo:** `main.js` importa el módulo, encadena operaciones, guarda, recarga y comprueba en *Application*.

Misión 1 · Criterios de aceptación y retos

✅ No está terminada hasta marcar todo:

- ☐ Ninguna función muta sus argumentos: `console.log` del original antes y después lo demuestra.
- ☐ `duracionTotal(cancionesIniciales)` devuelve `"21:25"`.
- ☐ `porGenero` agrupa en `clásica`, `rock` y `synthpop`.
- ☐ Modificar una instantánea de `structuredClone` no toca la lista original (ni sus objetos anidados).
- ☐ `cargar()` no lanza excepción aunque guardes basura a mano en `"aurora-fm"`.
- ☐ Todo en un módulo ES con `export` / `import`.

🚀 Retos extra

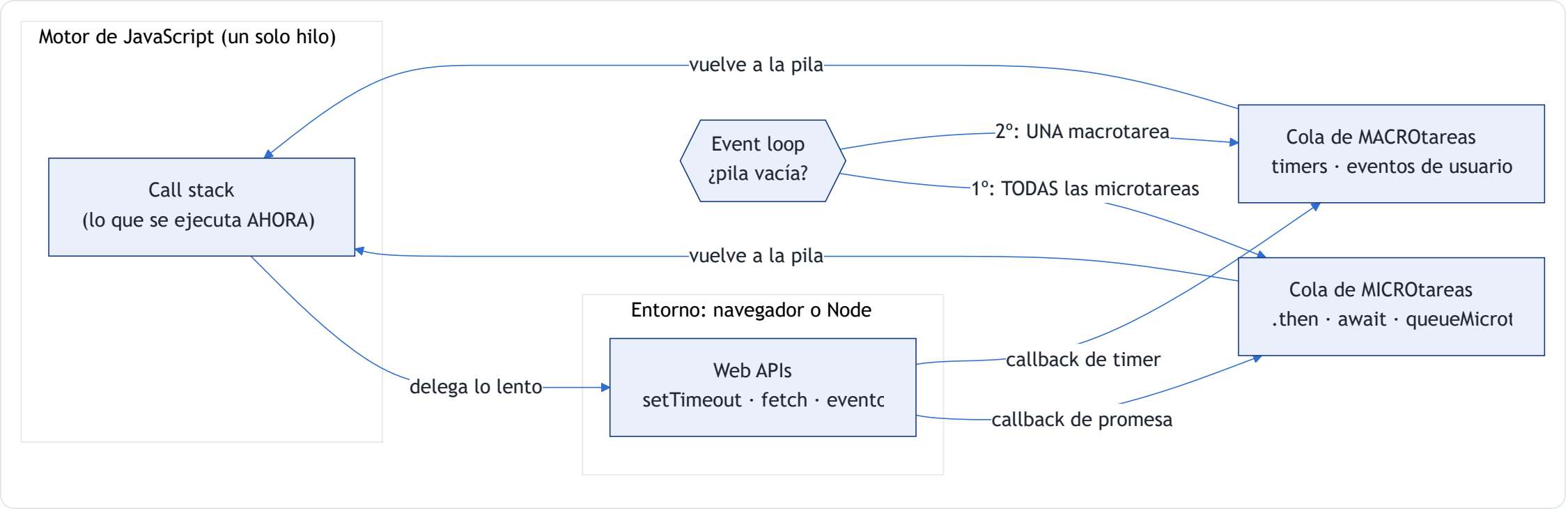
- `deshacer()`: historial de instantáneas en un array y recuperar la anterior.
- `buscar(lista, texto)` por título **o** artista, sin distinguir mayúsculas ni tildes (`normalize("NFD")`, `localeCompare`).

> Semáforo 🟡: pídele a la IA críticas y pistas («¿esta función muta algo?»), no la solución. El código lo escribes tú. Logro: **DJ residente**.

10 · Asincronía: el *event loop*, promesas y `async/await`

JavaScript tiene **un solo hilo**. Cómo no se bloquea, y en qué orden pasan las cosas.

El event loop



1. Lo síncrono corre en la **call stack**. 2. Lo lento (`fetch`, `setTimeout`) se **delega** al entorno. 3. Al acabar, su *callback* se encola: **microtareas** (promesas) o **macrotareas** (timers, eventos). 4. Con la pila vacía, el loop vacía **todas** las microtareas y luego toma **una** macrotarea.

DEMO EN VIVO

Predice el orden antes de ejecutar

```
console.log("1");                          // síncrono
setTimeout(() => console.log("2"), 0);     // macrotarea
Promise.resolve().then(() => console.log("3")); // microtarea
console.log("4");                          // síncrono

// Predice el orden ANTES de pulsar Ejecutar.
```

- Regla de oro: las **promesas** (microtareas) tienen prioridad sobre `setTimeout` (macrotarea), aunque el *timeout* sea `0`.
- 🧪 Experimento: añade un segundo `Promise.resolve().then(() => console.log("3 bis"))` justo después del primero. ¿Se cuela antes o después del `2`?

Ahora tú · 1.4 El oráculo del event loop

⏱ **15 min** · 🔴 **sin IA** · cuaderno 2, nivel 1

Sin ejecutarlo, escribe en papel el orden exacto de salida. Después ejecútalo y compara. Si fallas, explica por escrito qué cola (síncrona, microtareas o macrotareas) habías colocado mal.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve()
  .then(() => console.log("C"))
  .then(() => console.log("D"));
setTimeout(() => console.log("E"), 0);
console.log("F");
```

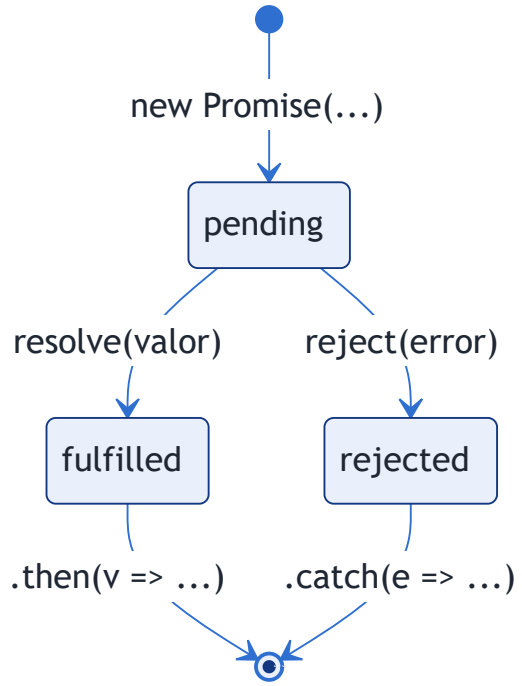
Salida esperada (no mires hasta haberlo escrito):

```
A F C D B E
```

> 💡 Primero se vacía la *call stack* (todo lo síncrono), después **todas** las microtareas (incluidas las que otras microtareas encolan, como el segundo `.then`) y solo entonces las macrotareas, en su orden de llegada.

Logro 🔮 **Oráculo asíncrono**: acertar sin ejecutar.

Promesas: un valor futuro con estado definitivo



Una vez sale de *pending*, el estado **no cambia**: un segundo `resolve()` no hace nada, y un `.then()` añadido tarde se ejecuta igualmente.

```
const promesa = new Promise((resolve, reject) => {
  const exito = true;
  if (exito) resolve("¡Datos listos!");
  else reject(new Error("Algo falló"));
});

promesa
  .then(resultado => console.log(resultado)) // si resolve
  .catch(error => console.error(error))      // si reject
  .finally(() => console.log("Terminado"));  // siempre

// Encadenar aplana el "callback hell":
obtenerUsuario(id)
  .then(usuario => obtenerPedidos(usuario))
  .then(pedidos => obtenerDetalle(pedidos[0]))
  .catch(error => console.error("Error en la cadena:", erro
```

Combinadores: varias promesas a la vez

```
const p1 = Promise.resolve(1);
const p2 = Promise.resolve(2);
const p3 = Promise.reject(new Error("fallo"));

await Promise.all([p1, p2]);        // [1, 2]
await Promise.race([p1, p2]);       // 1
await Promise.allSettled([p1, p3]);
// [{status:"fulfilled", value:1},
//  {status:"rejected", reason:Error}]
await Promise.any([p3, p2]);        // 2
```

> `await` de nivel superior solo en **módulos ES**; en un script clásico, dentro de una función `async`.

Combinador	Resultado
`all`	Todas OK → array; **una falla → rechaza** ya
`allSettled`	Siempre resuelve, con el estado de cada una
`race`	La **primera** en terminar (resuelva o rechace)
`any`	La primera en **resolverse**

- Equipo de 6 Pokémon: `all` (o `allSettled` si alguno puede fallar).
- *Timeout*: `race` contra un `setTimeout` que rechaza.

`async/await`: asincronía con aspecto síncrono

```
async function cargarDetalle(id) {
  try {
    const usuario = await obtenerUsuario(id);
    const pedidos = await obtenerPedidos(usuario);   // depende del anterior: secuencial OK
    const detalle = await obtenerDetalle(pedidos[0]);
    return detalle;
  } catch (error) {
    console.error("Error:", error.message); // captura cualquier rechazo de arriba
  } finally {
    console.log("Carga finalizada");        // apagar el «cargando», pase lo que pase
  }
}
```

- Una función `async` **siempre devuelve una promesa**; `await` pausa esa función (no el hilo) hasta que la promesa se resuelve.
- 💡 Operaciones **independientes**: no las pongas en `await` seguidos; lánzalas con `Promise.all`: `const [a, b] = await Promise.all([cargarA(), cargarB()]);`

DEMO EN VIVO

Secuencial o en paralelo: cronometrado

```
const tardar = (ms, valor) => new Promise(r => setTimeout(() => r(valor), ms)); // petición simulada

async function comparar() {
  let t0 = performance.now();              // ❌ Secuencial: cada await espera al anterior
  const a1 = await tardar(400, "A");
  const b1 = await tardar(400, "B");
  const c1 = await tardar(400, "C");
  console.log(`Secuencial: ${Math.round(performance.now() - t0)} ms →`, a1 + b1 + c1);

  t0 = performance.now();                  // ✅ En paralelo: las tres salen a la vez
  const [a2, b2, c2] = await Promise.all([tardar(400, "A"), tardar(400, "B"), tardar(400, "C")]);
  console.log(`Promise.all: ${Math.round(performance.now() - t0)} ms →`, a2 + b2 + c2);
}
comparar();
```

> ⚠️ Solo si las operaciones son **independientes**. Si la segunda necesita un dato de la primera, el `await` secuencial es lo correcto: hay que saber decirlo.

DEMO EN VIVO

Los tres estados de una carga: cargando, error, datos

```
<!doctype html>
<html lang="es">
<body style="font-family: system-ui; padding: 8px">
  <button id="ok">Cargar bien</button>
  <button id="mal">Provocar un error</button>
  <div id="salida" style="margin-top: 12px; min-height: 3em"></div>
  <script>
    const salida = document.getElementById("salida");
    // Simula una petición que tarda y que a veces falla
    const pedirDatos = (fallar) =>
      new Promise((resolve, reject) =>
        setTimeout(() => (fallar ? reject(new Error("HTTP 500")) : resolve(["Ana", "Leo", "Mar"])), 9
      );
    async function cargar(fallar) {
      salida.innerHTML = "<p>⏳ Cargando…</p>";          // 1. estado de carga
      try {
```

11 · La API `fetch`

Peticiones HTTP con promesas. Lo esencial aquí; a fondo en la Unidad 4.

DEMO EN VIVO

`fetch` no rechaza con un 404: comprueba `respuesta.ok`

```
async function obtenerPokemon(nombre) {
  const respuesta = await fetch(`https://pokeapi.co/api/v2/pokemon/${nombre}`);
  // fetch solo rechaza ante fallos de RED; un 404 o un 500 es una respuesta "válida":
  if (!respuesta.ok) throw new Error(`HTTP ${respuesta.status} al pedir "${nombre}"`);
  const datos = await respuesta.json();   // parsea el cuerpo JSON (también asíncrono)
  return { nombre: datos.name, id: datos.id, tipos: datos.types.map(t => t.type.name) };
}

async function demo() {
  try {
    console.log(await obtenerPokemon("pikachu"));
    console.log(await obtenerPokemon("misingno")); // 404 → al catch gracias al throw
  } catch (error) { console.error("Fallo:", error.message); }
}
demo();
```

Misión 2 · «Pokédex de campo»: asincronía en estado salvaje 🎓

🟡 **IA como pair · ⏱ 90 min** · cuaderno 2, nivel 2

- 🎯 **Objetivo:** un módulo asíncrono que consulte Pokémon reales (PokéAPI), tolere fallos de red y cargue equipos en paralelo.
- 🧰 **Practicas:** `fetch`, `async/await`, `try/catch`, errores personalizados, `Promise.all`, `allSettled` y `race`.

```
const BASE = "https://api.pokeapi.co"; // ⚠️ URL incorrecta a

export class ErrorPokedex extends Error {
  constructor(mensaje, codigo) {
    super(mensaje);
    this.name = "ErrorPokedex";
    this.codigo = codigo;
  }
}
```

Fases

1. **Consulta básica:** `obtenerPokemon(nombre)` → `{ nombre, id, tipos, peso }`, comprobando `respuesta.ok`.
2. **Errores con clase:** lanza `ErrorPokedex` con `codigo` cuando no sea `ok`; quien llama distingue con `instanceof` un 404 de un fallo de red.
3. **Equipo en paralelo:** `obtenerEquipo(nombres)` con `Promise.all`; mide con `console.time` frente a la versión secuencial.
4. **Tolerante a fallos:** `obtenerEquipoSeguro` con `allSettled` → `{ capturados, escapados }`.
5. **Paciencia limitada:** `conTimeout(promesa, ms)` con `Promise.race`; nada tarda más de 4 s.

Misión 2 · Criterios de aceptación y retos

✅ No está terminada hasta marcar todo:

- ☐ `obtenerPokemon("pikachu")` resuelve con `{ nombre: "pikachu", id: 25, tipos: ["electric"], … }`.
- ☐ `obtenerPokemon("misingno")` rechaza con un `ErrorPokedex` de `codigo` 404 (no con un `TypeError`).
- ☐ `Promise.all` con un equipo de 6 es claramente más rápido que la versión secuencial (mediciones en un comentario).
- ☐ `obtenerEquipoSeguro(["pikachu", "misingno", "eevee"])` **resuelve** con 2 capturados y 1 escapado.
- ☐ `conTimeout` funciona con cualquier promesa, no solo con las de la Pokédex.
- ☐ Ningún error acaba como *unhandled rejection* en la consola.

🚀 Retos extra

- **Caché** en un `Map` para no repetir peticiones (te servirá en el reto jefe).
- `Promise.any` para consultar la especie en dos idiomas y quedarte con la primera.
- `AbortController`: cancelar de verdad la petición al expirar el *timeout*.

> Pistas: `all` rechaza en cuanto una falla, por eso el equipo «seguro» usa `allSettled`; en `allSettled` cada resultado tiene `status` y `value` / `reason`.
> Logro: **Maestra/o Pokémon**.

12 · Herramientas: npm, Vite y TypeScript

Del fichero suelto al proyecto profesional. Lo que la Unidad 3 da por sabido.

npm y Vite: el andamio de un proyecto

```
npm init -y           # crea package.json (manifie
npm install zod       # dependencia → dependencies
npm install -D vite   # solo desarrollo (-D)
npm run dev           # script de package.json
```

- `package.json`: proyecto, dependencias y *scripts*.
- `node_modules/`: paquetes descargados. **Nunca** al repo (`.gitignore`).
- `package-lock.json`: versiones exactas, proyecto reproducible.
- Alternativas más rápidas: **pnpm**, **bun** (mismos comandos).

```
npm create vite@latest mi-proyecto  # elige Vanil
cd mi-proyecto
npm install
npm run dev     # http://localhost:5173, recarga
npm run build   # versión optimizada en /dist
```

- Un *bundler* transforma y combina módulos, TypeScript o JSX en algo que el navegador entiende.
- **Vite 8** (2026): un único empaquetador en Rust, **Rolldown**, para desarrollo y producción.
- La Arena comprueba en M2 que `vite` está en `package.json` (check de tecnología).

TypeScript: por qué tipos, y los tipos básicos

```
function duplicar(n) {   // JavaScript: acepta cualquier
  return n * 2;
}
duplicar("hola"); // NaN en ejecución, sin avisar 😱
```

```
function duplicar(n: number): number { // TypeScript
  return n * 2;
}
duplicar("hola"); // ❌ Error en el editor:
// 'string' no es asignable a 'number'
```

Todo JS válido es TS válido. Las anotaciones se comprueban **antes** de ejecutar y luego desaparecen: el navegador ejecuta JavaScript normal.

```
let nombre: string = "Ana";
let edad: number = 30;
let activo: boolean = true;
let etiquetas: string[] = ["js", "ts"];  // array
let punto: [number, number] = [10, 20]; // tupla

// Unión: uno u otro
let id: string | number = 42;
type CampoOrden = "titulo" | "duracion" | "artista";

// Inferencia: TS deduce el tipo
let ciudad = "Madrid"; // string, sin anotar
```

> Estándar del frontend profesional en 2026 (React, Next.js). TypeScript 7 es 10× más rápido; los conceptos son los mismos.

Interfaces, `readonly` y genéricos

```ts
interface Usuario {
  id: number;
  nombre: string;
  email?: string;          // opcional
  readonly creado: Date;   // solo lectura
}

const u: Usuario = { id: 1, nombre: "Lucía", creado: r

function saludar(usuario: Usuario): string {
  return `Hola, ${usuario.nombre}`;
}
```

```ts
// Genérico: reutilizable con distintos tipos, sin per
function primero<T>(lista: T[]): T | undefined {
  return lista[0];
}
primero<number>([1, 2, 3]); // number
primero(["a", "b"]);         // string (inferido)

// Frontera de confianza: JSON.parse devuelve any
const datos = JSON.parse(texto) as Cancion[];
```

- En React tiparás *props*, estado (`useState<number>`) y respuestas del API. No hace falta más para empezar.
- **Cero** `any` : es criterio de la Misión 3 y de la rúbrica del jefe.

Misión 3 · «Operación TypeScript»: la gran migración 🛡️

🟡 **IA como pair · ⏱️ 75 min** · cuaderno 2, nivel 2

- 🎯 **Objetivo:** migrar el módulo de Aurora FM a un proyecto **Vite (Vanilla + TypeScript)** completamente tipado: el compilador detecta los errores antes de ejecutar.
- 🧰 **Practicas:** npm, `package.json`, Vite, tipos básicos, `interface`, uniones, `readonly` y genéricos.

Fases

1. **El andamio:** `npm create vite@latest aurora-fm`, `npm run dev`; localiza los scripts y el `.gitignore`.
2. **La forma de los datos:** `src/tipos.ts` con `interface Cancion` (`readonly id`) y `type CampoOrden`.
3. **Migración:** `playlist.js` → `src/playlist.ts`, **todas** las firmas anotadas.
4. **Rompe cosas a propósito:** `ordenarPor("precio")`, reasignar `id`, canción sin `duracion`. Anota el mensaje exacto del editor.
5. **Un genérico propio:** `ultima<T>(lista: T[]): T | undefined` con `Cancion[]` y con `string[]`.

✅ Criterios

- ☐ `npm run dev` levanta y `npm run build` compila sin errores.
- ☐ Todas las funciones públicas con tipos explícitos en parámetros y retorno.
- ☐ Los tres errores provocados aparecen subrayados **antes** de ejecutar.
- ☐ `ultima` funciona con dos tipos distintos sin `any`.
- ☐ Ningún `any` en el proyecto.

🚀 **Retos:** `porGenero` como `Partial<Record<string, Cancion[]>>` · `type Resultado<T> = { ok: true; valor: T } | { ok: false; error: string }` para `cargar()`.

> Si el editor no marca errores: abre la **carpeta del proyecto** (donde vive `tsconfig.json`), no un fichero suelto. Logro: **Guardián de tipos**.

Buenas prácticas modernas

- ✅ **Inmutabilidad** por defecto: `const`, `toSorted` / `with`, *spread* en vez de mutar.
- ✅ `map` / `filter` / `reduce` en vez de bucles manuales cuando aporten claridad.
- ✅ `async/await` con `try/catch` en vez de cadenas largas de `.then()`.
- ✅ Comprueba `respuesta.ok` en cada `fetch`; muestra cargando, error y vacío.
- ✅ Lanza objetos `Error` (o subclases de dominio), nunca strings.
- ✅ `structuredClone` para copias profundas; cuidado con los anidados del *spread*.
- ✅ Desestructura parámetros y resultados; funciones **puras** siempre que puedas.
- ✅ Separa **API, lógica y render** en módulos; tipa las fronteras.
- ⚠️ `await` en fila con operaciones independientes; `await` dentro de `forEach`.
- ⚠️ `sort()` / `reverse()` sobre un array que no es tuyo.

Reto jefe · «Atlas multiverso»: tu primera SPA sin frameworks 👑

🟢 **IA libre (declarada)** · ⏱️ **4–5 h** · cuaderno 2, nivel 3

Catálogo de personajes de Rick and Morty (`rickandmortyapi.com/api/character`) en **Vite + TypeScript**, con caché en memoria y favoritos persistentes. Integra **todo** lo anterior.

Requisitos

1. **Búsqueda:** `<input>` + botón → `?name=<texto>` → tarjetas (nombre, imagen, especie, estado) generadas desde JS.
2. **Arquitectura:** `class ClienteApi` (o *factory*) con `fetch`, `respuesta.ok` y **caché en** `Map` (clave: la URL). Misma búsqueda dos veces = una petición (*Network*).
3. **Errores visibles:** 404 o fallo de red → mensaje amable, nunca consola en rojo.
4. **Favoritos:** ⭐ por tarjeta, en `localStorage`, restaurados al recargar.
5. **Estadísticas:** recuento por especie con `Object.groupBy`; favoritos con `toSorted`.
6. **Tipos:** `Personaje` y `RespuestaApi`; cero `any`.

```
atlas-multiverso/
├── index.html
└── src/
    ├── main.ts     # arranque y eventos
    ├── tipos.ts    # Personaje, RespuestaApi
    ├── api.ts      # ClienteApi: fetch + Map
    ├── estado.ts   # favoritos + localStorage
    └── ui.ts       # tarjetas, errores, stats
```

Checkpoints: datos en consola → tarjetas → caché verificada en *Network* → favoritos tras `F5` → estadísticas.

🚀 **Retos:** paginación (`info.next`/`prev`) · enrutado con `history.pushState` · `import()` dinámico del panel de estadísticas.

Reto jefe · Rúbrica de autoevaluación (18+ es nivel jefe)

Criterio	0 puntos	3 puntos	5 puntos
Asincronía y errores	Solo el camino feliz	`try/catch` y `respuesta.ok` en cada petición	Además: *timeout* o cancelación, y mensajes de error diferenciados en la UI
Caché con `Map`	No hay caché	La caché evita peticiones repetidas	Además: expiración por tiempo o límite de tamaño
Inmutabilidad y datos	Mutaciones por todas partes	Transformaciones con métodos inmutables	Además: estado centralizado, funciones puras y `structuredClone` donde toca
TypeScript	Tipos mínimos o `any` sueltos	Interfaces para API y funciones tipadas	Además: uniones, genéricos o utilidades (`Partial`, `Record`) bien empleados
Código y estructura	Todo en un fichero	Módulos separados (api, estado, UI) con `import` / `export`	Además: nombres cuidados, sin código muerto y `npm run build` limpio

> Las cinco filas son las mismas que puntúa la Arena en M2 (Asincronía, Transformación de datos, Módulos, Robustez): el reto jefe **es** un candidato a entrega.

Hacia la Arena · Misión M2 «Async Odyssey»

El event loop no espera a nadie.

Qué se entrega: una URL de GitHub con una app hecha **con Vite** que consuma una **API pública real** (PokéAPI, TheMealDB, OpenLibrary…) usando `fetch` + `async/await`, módulos ES y métodos de array, y pinte los datos en el DOM con **estados de carga y error**.

Cómo se puntúa (/100)

- **30** checks automáticos: README, repo limpio, ≥ 5 commits, tecnología (`vite` en `package.json`, `.js` / `.ts`), estructura.
- **70** rúbrica evaluada por IA: Asincronía **20** · Transformación de datos **15** · Módulos y estructura **15** · Robustez **10** · Originalidad **10**.
- 🎁 Bonus: **caché en** `localStorage` para no repetir peticiones ya hechas.

La nota es provisional hasta la defensa oral. La IA genera 3 preguntas sobre **tu** código; en el ranking cuenta el mejor intento.

- README con sección **«Uso de IA»**: qué pediste, qué cambiaste, qué entiendes.
- `try/catch` + `respuesta.ok` en cada petición; cargando, error y vacío **visibles**.
- API, lógica y render en módulos distintos; `map` / `filter` / `reduce`, no bucles a mano.
- `node_modules/` y `dist/` fuera del repo.

> Objetivos oficiales: `fetch` con `async/await` y errores · `map` / `filter` / `reduce` antes de pintar · módulos ES (API, lógica, render) · proyecto Vite con `package.json` coherente.

Entrena la defensa oral

Preguntas del mismo corte que las de la Arena. Se responden en 1–2 minutos **si has entendido**; de memoria son imposibles.

1. Un `console.log`, un `setTimeout(…, 0)` y un `Promise.resolve().then(…)`, en ese orden. El *timeout* es de 0 ms y aun así va el último: ¿por qué? ¿Qué tendría que pasar para que el `.then()` se retrasara?
2. Haces un `fetch` dentro de un `try/catch`. Si el servidor responde **404**, ¿entra en el `catch`? ¿Qué comprobación añades y por qué el `try/catch` solo no basta?
3. Tu app carga tres recursos independientes: diferencia entre tres `await` seguidos y `Promise.all`. ¿En qué caso los tres `await` seguidos serían lo correcto?
4. Ordenas resultados para pintarlos: ¿por qué `toSorted()` y no `sort()`? ¿Qué problema concreto aparece con `sort()` al llegar a React?
5. `const copia = { ...config }` y al cambiar `copia.tema.color` cambia también `config.tema.color`. ¿Por qué? ¿Cómo lo arreglas sin `JSON.parse(JSON.stringify(…))`?
6. La misión pide separar API, lógica y render. Si lo metes todo en un `main.js` de 300 líneas, ¿qué se rompe **exactamente**? No vale «queda más ordenado».

> Las ocho preguntas con respuesta modelo están en la **autocomprobación** de la unidad, en la Arena (`/teoria`).

Tabla de logros del cuaderno 2

Logro 🏅	Cómo desbloquearlo
🥋 **Cinturón inmutable**	Los 4 ejercicios del Nivel 1 resueltos y comprobados
🔮 **Oráculo asíncrono**	Acertar el orden del 1.4 **sin** ejecutar el código
📻 **DJ residente** · 🎓 **Maestra/o Pokémon** · 🛡️ **Guardián de tipos**	Terminar cada misión con todos los criterios marcados (la 3, sin ningún `any`)
⚡ **Cazadora/or de latencia**	Mediciones de `Promise.all` frente a secuencial documentadas
🧠 **Memoria fotográfica**	La pestaña *Network* demuestra que la caché evita peticiones
👑 **Jefe del multiverso**	18 o más puntos en la rúbrica del reto jefe
🌟 **Leyenda de Aurora**	Todos los retos extra de las tres misiones completados

Recursos y próxima unidad

Para profundizar

- MDN — Array (métodos): https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array
- MDN — Closures: https://developer.mozilla.org/es/docs/Web/JavaScript/Closures
- MDN — Promesas y `async` / `await`: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Promise
- javascript.info — El event loop y microtareas: https://javascript.info/event-loop
- MDN — `fetch`: https://developer.mozilla.org/es/docs/Web/API/Window/fetch
- MDN — Módulos JavaScript: https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Modules
- Vite — Guía de inicio: https://vite.dev/guide/
- TypeScript para programadores de JavaScript: https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html

Unidad 3 — React Core

- Componentes, JSX y *props*: la desestructuración de esta unidad en acción.
- `useState`: un closure que recuerda; el estado se actualiza **sin mutar** (`{ ...obj }`, `toSorted`).
- `useEffect` y su limpieza (`clearInterval`), listas con `key`, renderizado condicional.
- Proyecto con Vite + React + TypeScript.
- Misión M3 «Reactor Core»: reconstruir tu app de M2 en React.